

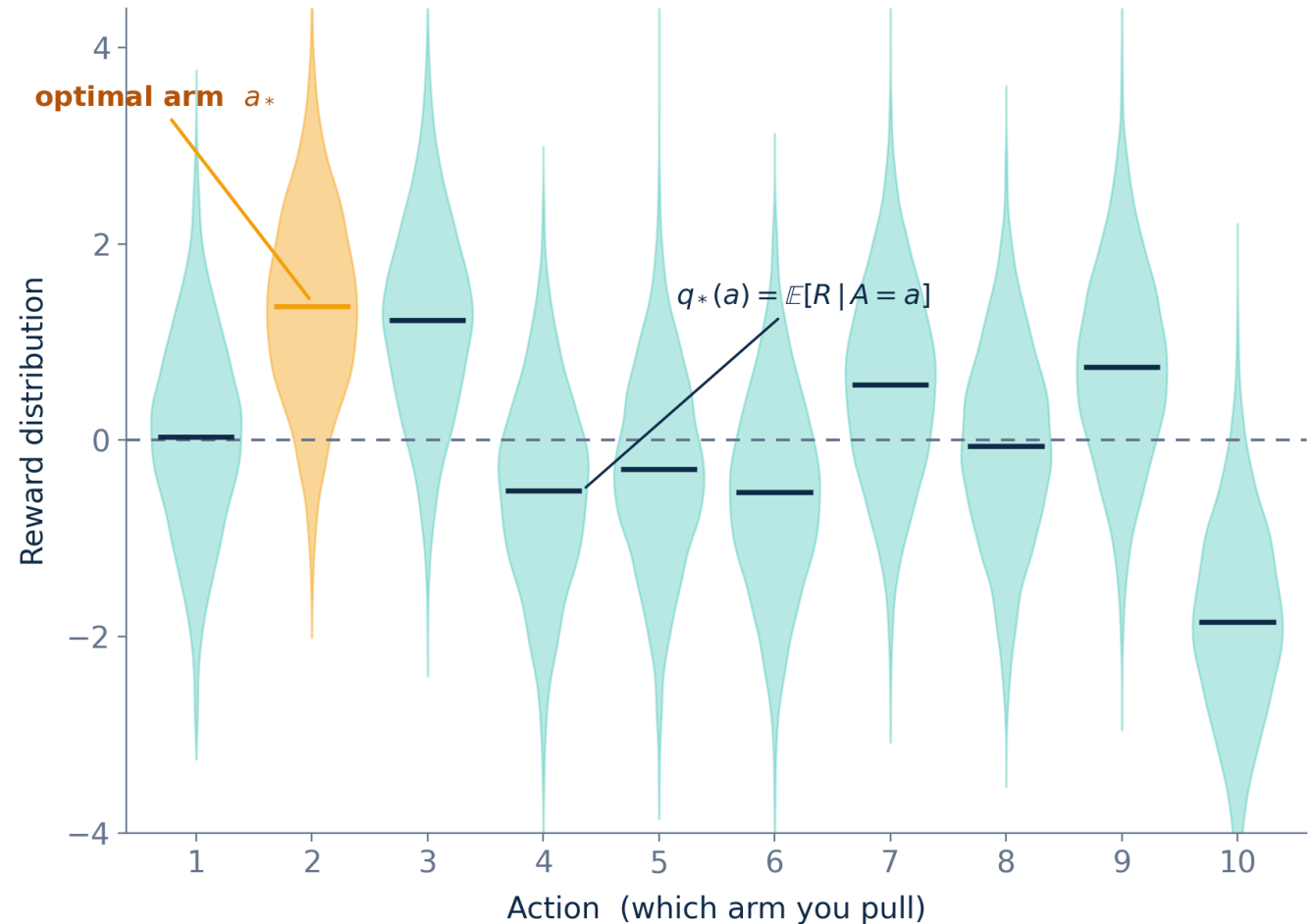
The K-Armed Bandit

What you're given

- K arms; pulling arm a returns a reward R_t .
- Each arm has a fixed but unknown reward distribution.
- Its mean $q^*(a) = \mathbb{E}[R_t \mid A_t = a]$ is what you'd love to know.
- You only see the reward you collected — never the others.
- Goal: maximize total reward over many pulls.

The catch

$q^*(a)$ is hidden. You must estimate it from noisy rewards while you act — so pulling an arm to **learn (explore)** competes with pulling the best-looking arm to **earn (exploit)**.



Each arm's reward is a noisy draw; the bar marks its true value $q^*(a)$. **Arm a^* has the highest mean** — but you don't know that yet.

Estimate each action's value, one pull at a time.

Sample mean estimate:

$$\hat{Q}(a) = (\text{sum of rewards from } a) \div (\text{\# pulls of } a)$$

Equivalent incremental update — no need to store all data:

$$\hat{Q}_{\text{new}}(a) = \hat{Q}_{\text{old}}(a) + \alpha \cdot (r - \hat{Q}_{\text{old}}(a))$$

new estimate = old estimate + step size · prediction error

$\alpha = 1/n$ → exact running average $\alpha = \text{constant}$ → exponentially-weighted average (adapts to change)

ϵ -greedy: explore randomly, exploit greedily.

Algorithm

With probability $1 - \epsilon$:
 choose $\text{argmax}_a Q(a)$ (exploit)
With probability ϵ :
 choose a uniformly at random (explore)



■ exploit ($1 - \epsilon$) ■ explore (ϵ)

Pros

- Dead simple
- Strong baseline
- Hard to beat at small scale

Cons

- Explores blindly
- Keeps trying bad actions forever (unless ϵ decays)
- Ignores uncertainty